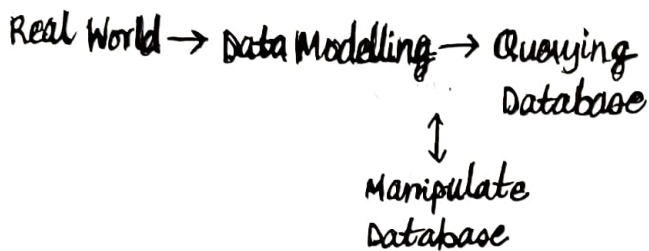


7.1 Manipulating Databases

We must carefully manipulate a database to be consistent and up to date with the real world.

Data Management Process



Basic Functions of DBMS: CRUD

- Create new data elements
- Read data by asking questions
- Update data elements
- Delete data elements.

Data Elements: Database, table, schema, tuple, attribute.

Our view: CRUD = Manipulation (CRU) + Querying (D)

Manipulating Database: By Purposes

- Organizing Databases
 - create and delete data "spaces" - database, schema, table.
- Modifying Databases
 - Insert, delete, update tuples to a table
- Augmenting Databases
 - create supporting (redundant) data elements: Indexes and views.
- Regulating Databases
 - Creating constraints (rules) that must maintain true all the time.

How do we perform manipulations?

- System
 - Console software: DBMS native interface via shell programs.
 - SQL clients: Using SQL for manipulation.
- Language
 - Every DBMS provides a language for manipulating/querying it
- For relational DB (SQL)
 - Querying commands: SELECT-WHERE-FROM, GROUP BY, ...
 - Manipulating commands: CREATE, INSERT, UPDATE, DROP, DELETE

7.2 Organizing Databases

Organize Our Space of Data: Database

- A space of data that we manage for our applications
- A set of tables that capture the data in your application domain.
- Different names or non-standard meanings at different systems.

Database Organization

Database ($\rightarrow$ Schema) $\rightarrow$ Table

- Most databases support 3 levels:
 - Database $\rightarrow$ Schema $\rightarrow$ Table
 - E.g. Oracle, PostgreSQL
 - The term "schema" is overloaded
- Some support 2 levels:
 - Database $\rightarrow$ Table
 - E.g. MySQL

Organizing Databases

- MySQL as example.
- Create a database
`CREATE DATABASE <name>;`
- Show all databases
`SHOW DATABASES;`
- Delete a database
`DROP DATABASE <name>;`
- Select a database
`USE <name>;`

Organizing Tables

(43)

- Create a table
`CREATE TABLE <name>
(<elements>);`
- Show all tables
`SHOW TABLES;`
- Show the schema of a table
`DESCRIBE TABLE <name>;`
- Change the schema
`ALTER TABLE <name>
ADD/DROP <attribute>;`
- Delete a table
`DROP TABLE <name>;`

Creating a Table

- We create a table by declaring its schema.
`CREATE TABLE <name> (<elements>);`
- Element: Attribute + constraint:
 - Attribute: Name, type
 - E.g.: bar char(30)
 - Constraint: Optional, including
 - PRIMARY KEY, UNIQUE KEY, FOREIGN KEY
 - NOT NULL
 - DEFAULT <value>
- Key constraints can be separate elements.
 - Necessary for multi-attribute keys.
 - E.g.: PRIMARY KEY (bar, beer)

Constraints are part of Database Creation

- A constraint is a relationship among data elements that the DBMS is required to enforce.
- Types of constraints:
 - Key constraints: Constrain attributes to serve as some "keys" (identifiers) -- primary key, unique key, foreign key.
 - Attribute-based constraints:
 - Constrain values of an attribute to meet some criteria.
 - Tuple-based constraints:
 - Constrain relationship among components in a tuple.
 - Database-based constraints:
 - Constrain relationship among elements ~~any~~ in any table in the database.

Changing Schema of Table

- Add attributes
`ALTER TABLE <name>
ADD <attribute declaration>;`
 - Delete attributes
`ALTER TABLE <name>
DROP COLUMN <attribute>;`
 - Modify attributes
`ALTER TABLE <name>
MODIFY COLUMN <attribute declaration>;`
-
- `ALTER TABLE sells ADD size int NOT NULL`

7.3 Modifying Databases

(44)

Modifying a Table

- We modify a database by
 - Insertion: Add a tuple
 - Deletion: Delete a tuple
 - Update: Modify a tuple

Insertion with Values or Query Results

- Insert tuples with some or all attribute values.

`INSERT R (A1, ..., An)` ← Relation and attributes to insert into.

`VALUES (v11, ..., v1n), ..., (vm1, ..., vmn);`
↳ values to insert

- Can omit attribute names if the same as in schema definition.

- Insert the results of queries

`INSERT R (A1, ..., An)` ← Relation and attributes to insert into

`(SUBQUERY);` ← Query results to insert

e.g.

`INSERT INTO Beers(name, brewer) VALUES ('Goose Island IPA', 'Goose Island');`

`INSERT INTO Beers(name) SELECT beer FROM Sells s WHERE NOT EXISTS (SELECT * FROM Beers WHERE name = s.beer);`

Deletion

- DELETE tuples that match some condition.

`DELETE FROM <name>
WHERE <condition>;`

- If WHERE is omitted: delete all!

`DELETE FROM <name>;`

Updates

- Change certain attributes in certain tuples of a relation:

UPDATE <name>

SET <list of attribute assignments>

WHERE <condition>;

e.g. UPDATE drinkers SET price = 5
WHERE name = 'Dan'

UPDATE sells SET price = price * 1.3
WHERE price < 3;

UPDATE sells s

SET price = 0

WHERE price =

(SELECT MAX(price) FROM sells WHERE
beer = s.beer)

standard semantics of modification: "deferred" updates

1. Evaluate WHERE expression and identify every tuple that needs to be updated.
2. Update the tuples identified.

* The <value expression>s are effectively evaluated before updating the object row.

7.4 Augmenting Databases: Indexes (45)

Indexes: What and Why?

- An index is a "map" of a table, telling DBMS where different values are stored.
 - E.g. SELECT sid, grade FROM Enrolls
WHERE number = "CS411" AND
grade = 'A';
 - How to locate tuples with number "CS411"? Where are those "A" tuples?
- We need indexes for those attributes that are frequently used in queries, so that DBMS can process queries quickly.
 - But DBMS can still execute (maybe slower) without indexes.
- Users do not view or access indexes directly.
 - They are used by "query processor" in DBMS internally.

Creating an Index

- DBMS may create an index automatically for important attributes.
 - MySQL: The "PRIMARY" index, auto created from PRIMARY KEY attributes.
- Users can tell DBMS what attributes to CREATE index for, since users anticipate what queries will be asked.

Manipulating Indexes

- Most DBMS provides ways for users to create indexes.
 - But syntax may be slightly different.
We use MySQL here.
- Create an index: `CREATE INDEX <name> ON <table> (<attributes>);`
- Show indexes:
`SHOW INDEX FROM <table>;`
- Delete an index:
`ALTER TABLE <table>
DROP INDEX <name>;`
- e.g. `CREATE INDEX price_idx ON sells(price);`

How will DBMS use Indexes?

- DBMS will use indexes to construct more efficient 'query plans'

`EXPLAIN SELECT bar, beer FROM sells WHERE price < 3;`

7.5 Augmenting Databases:

(46)

Views

Views: Our "Perspectives"!

- Our data modeling created tables to store in DB - the base tables - to capture the essential information of the application domain.
- However, different users may have different "perspectives", and they may want a particular subset organized in particular ways.
 - AcademicWorld:
 - Roles: students vs. instructors vs. administrators.
 - scope/web: CS department vs EE dept. vs College of Engineering
- Thus, we define a view to capture a "perspective" -- a virtual table that is defined in terms of base tables (and views).
- Views help to provide such different perspectives.

Creating and Querying a View

- Declare by
`CREATE VIEW <name> AS <query>;`
- View definition: the <query> statement, which can use base tables or other views.
- You can "query" a view as if it were a base table.
 - But a view may not be "updatable".

e.g. CREATE VIEW GreenStDrinkers AS
SELECT * FROM Drinkers WHERE addr =
"Green St";

Querying a View:

What happens when a view is used?

- View Expansion: DBMS will replace a view with its definition, thus turning the query into one with only base tables.
 - It will first rewrite the query and view definition both into some internal expressions, e.g., in relational algebra.
 - The view definition expression is then "sliced into" the query expression.

- SELECT name FROM GreenStDrinkers g
WHERE g.frequent = "Green Bar";
 - Query: $Q = \pi_{name}(\sigma_{frequent = "Green Bar"}(GreenStDrinkers))$
 - View: $GreenStDrinkers = \sigma_{addr = "Green St"}(Drinkers)$

• Query with View Expanded:

$Q' = \pi_{name}(\sigma_{frequent = "Green Bar"}(\sigma_{addr = "Green St"}(Drinkers)))$

Updating a View

- Can we insert a tuple into a "virtual" table?
- Not all views are "updatable"?

aren't

Why ~~are~~ All Views are Updatable? 47

- Values of a view may not sufficiently determine values of its base tables.
- 1. Values may be missing.
 - CREATE VIEW BarHasBeer AS (SELECT bar, beer FROM sells)
 - INSERT ("Green Bar", "Goose Island IPA")
 - What is price? Note that price must be NOT NULL in sells.
- 2. Values cannot not be uniquely determined.
 - CREATE VIEW DeptGPA AS (SELECT major, AVERAGE(gpa) FROM students GROUP BY major).
 - INSERT ("CS", 3.0)
 - How to map 3.0 average GPA to every student in "CS"?
- Views from joins are more complicated, involving multiple base tables.
 - Cannot uniquely determine the values for each base table.

Not all Views are "Updatable"

- Complex rules to determine if a view is updatable, but intuitively:

A view is updatable if it selects some enough attributes from one relation.

- A view that joins multiple tables are not updatable.
- A view with too few attributes is not updatable.